

Functional verification

Functional verification is the task of verifying that a logic design conforms to specification.^[1] Functional verification attempts to answer the question "Does this proposed design do what is intended?"^[2] This is complex and takes the majority of time and effort (up to 70% of design and development time)^[1] in most large electronic system design projects. Functional verification is a part of more encompassing *design verification*, which, besides functional verification, considers non-functional aspects like timing, layout, and power.^[3]

Background

Although the number of transistors increased exponentially according to Moore's law, the number of engineers and time taken to produce the designs only increased linearly. As the transistors' complexity increases, the number of coding errors also increases. Most of the errors in logic coding come from careles coding (12.7%), miscommunication (11.4%), and microarchitecture challenges (9.3%).^[1] Thus, electronic design automation (EDA) tools are produced to catch up with the complexity of transistors design. Languages such as Verilog and VHDL are introduced together with EDA tools.^[1]

Functional verification is very difficult because of the sheer volume of possible test-cases that exist in even a simple design. Frequently there are more than 10⁸⁰ possible tests to comprehensively verify a design – a number that is impossible to achieve in a lifetime. This effort is equivalent to program verification, and is NP-hard or even worse – and no solution has been found that works well in all cases.

Verification process and strategy

--

The verification plan

A functional-verification project is guided by a verification plan. This is a foundational document that serves as a blueprint for the entire effort. It is a living document created early in the design cycle and is critical for defining scope and tracking progress. The plan typically defines:^[4]

- Verification scope:** A list of the design features and functions that need to be verified.
- Methodology:** The techniques (e.g., simulation, emulation, formal) and standardized methodologies (e.g., UVM) that will be used.
- Resources:** The engineering team, EDA tools, and computational infrastructure required.
- Success criteria:** Specific coverage goals that must be met to consider the verification complete.

Coverage metrics

To measure the completeness of the verification effort, engineers rely on coverage metrics.^[4] The process of achieving the predefined coverage goals is known as "coverage closure." There are two main types of coverage:

- Code coverage:** This measures how thoroughly the hardware description language (HDL) source code has been exercised during testing. It includes metrics like statement coverage, branch coverage, and toggle coverage.
- Functional coverage:** This measures whether the intended functionality, as described in the verification plan, has been tested. Engineers define specific scenarios or data values of interest, and the verification tool tracks whether these cases have been exercised.

Levels of abstraction in verification

Functional verification is a continuous process that is applied at different levels of design abstraction as a chip is developed. This hierarchical approach is necessary to manage the complexity of modern SoCs.^{[5][4]}

- Unit/block-level verification:** This is the most granular level, where individual design modules or "units" (e.g., a single FIFO, an ALU, or a decoder) are tested in isolation. The goal is to thoroughly verify the functionality of a small piece of the design before it is integrated into a larger system.^[5]
- Subsystem/IP-level verification:** At this stage, multiple units are integrated to form a larger, functional block, often referred to as a subsystem or an Intellectual Property (IP) core (e.g., a complete memory controller or a processor core). Verification at this level focuses on the combined functionality and the interactions between the integrated units. A common strategy at this stage is the use of behavioral models, which are high-level, functional representations of a block. These models simulate faster than detailed RTL code and allow verification to begin before the final design is complete, helping to formalize interface specifications and find bugs early.^[4]
- SoC/chip-level verification:** Once all subsystems and IP blocks are available, they are integrated to form the full system-on-a-chip (SoC). Functional verification at the chip level is focused on verifying the correct connectivity and interactions between all these major blocks. System-level simulations are run to prove the interfaces between ASICs and check complex protocol error conditions.^[4]
- System-level verification:** This is the highest level of abstraction, where the verified chip's functionality is tested in the context of a full system, often including other chips, peripherals, and software. Hardware emulation is a critical technique at this stage, as its high speed allows for the execution of real software, such as device drivers or even booting a full operating system on the design. This provides a "richness of stimulus" that is very difficult to replicate in simulation and is highly effective at finding system-level bugs.^[4]

Verification methodologies

Because exhaustive testing is impossible, a combination of methods is used to attack the verification problem. These are broadly categorized as dynamic, static, and hybrid approaches.

--

Dynamic verification (simulation-based)

Dynamic verification involves executing the design model with a given set of input stimuli and checking its output for correct behavior. This is the most-widely-used approach.^[1]

- Logic simulation:** This is the powerhouse of functional verification, where a software model of the design is simulated. A testbench is created to generate stimuli, drive them into the design, monitor the outputs, and check for correctness.
- Emulation and FPGA prototyping:** These hardware-assisted techniques map the design onto a reconfigurable hardware platform (an emulator or an FPGA board). They run orders of magnitude faster than simulation, allowing for more extensive testing with real-world software, such as booting an operating system.^[5]
- Simulation acceleration:** This uses special-purpose hardware to speed up parts of the logic simulation.

A modern simulation testbench is a complex software environment. Key components include a generator to create stimuli (often using constrained-random techniques), a driver to translate stimuli into pin-level signals, a monitor to observe outputs, and a checker (or scoreboard) to validate the results against a reference model.

--

Static verification

Static verification analyzes the design without executing it with test vectors:^[1]

- Formal verification:** This uses mathematical methods to prove or disprove that the design meets certain formal requirements (properties) without the need for test vectors. It can prove the absence of certain bugs but is limited by the state-space explosion problem.
- Linting:** This involves using HDL-specific versions of lint tools to check for common coding style violations, syntax errors, and potentially problematic structures in the code.

--

Hybrid techniques

These approaches combine multiple verification techniques to achieve better results. For example, formal methods can be used to generate specific tests that target hard-to-reach corner cases, which are then run in the more scalable simulation environment.^[6]

Components of simulated environments

A simulation environment is typically composed of several types of components:

- The **generator** generates input vectors that are used to search for anomalies that exist between the intent (specifications) and the implementation (HDL Code). This type of generator utilizes an NP-complete type of SAT solver that can be computationally expensive. Other types of generators include manually created vectors and graph-based generators (GBMs). Modern generators create directed-random and random stimuli that are statistically driven to verify random parts of the design. The randomness is important to achieve a high distribution over the huge space of the available input stimuli. To this end, users of these generators intentionally under-specify the requirements for the generated tests. It is the role of the generator to randomly fill this gap. This mechanism allows the generator to create inputs that reveal bugs not being searched for directly by the user. Generators also bias the stimuli toward design corner cases to further stress the logic. Biasing and randomness serve different goals and there are tradeoffs between them, so different generators have different mixes of these characteristics. Since the input for the design must be valid and many targets (such as biasing) should be maintained, many generators use the constraint satisfaction problem (CSP) technique to solve the complex testing requirements. The validity of the design inputs and the biasing arsenal are modeled. The model-based generators use this model to produce the correct stimuli for the target design.
- The **drivers** translate the stimuli produced by the generator into the actual inputs for the design under verification. Generators create inputs at a high level of abstraction, namely, as transactions or assembly language. The drivers convert this input into actual design inputs as defined in the specification of the design's interface.
- The **simulator** produces the outputs of the design, based on the design's current state (the state of the flip-flops) and the injected inputs. The simulator has a description of the design net-list. This description is created by synthesizing the HDL to a low gate level net-list.
- The **monitor** converts the state of the design and its outputs to a transaction abstraction level so it can be stored in a "score-boards" database to be checked later on.
- The **checker** validates that the contents of the score-boards are legal. There are cases where the generator creates expected results, in addition to the inputs. In these cases, the checker must validate that the actual results match the expected ones.
- The **arbitration manager** manages all the above components together.

Verification for specialized design domains

--

Low-power verification

Modern SoCs employ sophisticated power-management techniques to conserve energy, such as power gating and multiple voltage domains. Verifying the correct functionality of these low-power features is a major task that involves ensuring logic states are correctly isolated, retained, and restored during power-down and power-up sequences. This is typically managed by specifying the power intent in a standardized format, such as the Unified Power Format (UPF), which guides the verification tools.^[4]

--

Clock domain crossing (CDC) verification

Complex SoCs often contain multiple clock domains that operate asynchronously to one another. Passing data reliably between these domains is a common source of subtle hardware bugs. CDC verification focuses on identifying and ensuring the correctness of synchronizer circuits used at these asynchronous boundaries to prevent issues like metastability and data corruption. Specialized static analysis and formal verification tools are essential for comprehensive CDC verification.^[4]

Emerging trends

--

--

Machine learning in functional verification

Machine learning (ML) is being applied to various aspects of functional verification to improve efficiency and effectiveness. ML models can analyze large datasets from the verification process to identify patterns and make predictions. Key applications include:^[7]

- Automated test generation:** Guiding stimulus generation to create tests more likely to exercise unverified parts of the design.
- Bug prediction and localization:** Analyzing design data to predict error-prone modules or to assist in pinpointing the root cause of failures.
- Coverage analysis:** Optimizing the process of achieving coverage goals by predicting which tests will be most effective at closing remaining coverage holes, thereby reducing the length of regression tests.

--

Hardware security verification

As electronic systems become more integrated into critical applications (e.g., AI, automotive), ensuring hardware security has become a key part of verification. The process is now being adapted to detect security vulnerabilities in addition to functional bugs. This includes testing for threats such as:^[8]

- Hardware trojans: These are malicious, hidden modifications to the design that can create a backdoor or cause the system to fail under specific conditions. Verification must attempt to uncover this unintended and hostile functionality.
- Side-channel attacks: These are vulnerabilities where information is leaked through physical characteristics like power consumption or electromagnetic emissions. While traditionally a post-silicon concern, pre-silicon verification is now being used to analyze designs for susceptibility to such attacks.

See also

- Analog verification
- Cleanroom software engineering
- High-level verification

References

- Molina, A; Cadenas, O (8 September 2006). "Functional verification: approaches and challenges" (http://www.scielo.org.ar/scielo.php?script=sci_arttext&pid=S0327-07932007000100013). *Latin American Applied Research*. **37**. ISSN 0327-0793 (https://search.worldcat.org/issn/0327-0793). Archived (https://web.archive.org/web/20221016121732/http://www.scielo.org.ar/scielo.php?script=sci_arttext&pid=S0327-07932007000100013) from the original on 16 October 2022. Retrieved 12 October 2022.
- Rezaeian, Banafsheh; Rodrigues, Dr. Joachim; Rath, Alexander W. "Simulation and Verification Methodology of Mixed Signal Automotive ICs" (https://www.eit.lth.se/srapport.php?uid=667). *Lund University, Department of Electrical and Information Technology*.
- Stroud, Charles E; Change, Yao-Chang (2009). "CHAPTER 1 – Introduction" (https://www.sciencedirect.com/science/article/pii/B9780123743640500084). *Design Verification*. pp. 1–38. doi:10.1016/B978-0-12-374364-0.50008-4 (https://doi.org/10.1016%2FB978-0-12-374364-0.50008-4). ISBN 978-0-12-374364-0. Archived (https://web.archive.org/web/20221012041625/https://www.sciencedirect.com/science/article/pii/B9780123743640500084) from the original on 12 October 2022. Retrieved 11 October 2022.

18/09/2026, 22:42

Functional verification - Wikipedia

4. Mehta, Ashok B. (2018). *ASIC/SoC Functional Design Verification* (<https://link.springer.com/book/10.1007/978-3-319-59418-7>). doi:10.1007/978-3-319-59418-7 (<https://doi.org/10.1007%2F978-3-319-59418-7>). ISBN 978-3-319-59417-0.

5. Evans, Adrian; Silburt, Allan; Vrckovnik, Gary; Brown, Thane; Dufresne, Mario; Hall, Geoffrey; Ho, Tung; Liu, Ying (1998-05-01). "Functional verification of large ASICs" (<https://dl.acm.org/doi/10.1145/277044.277210>). *Proceedings of the 35th annual conference on Design automation conference - DAC '98*. New York, NY, USA: Association for Computing Machinery. pp. 650–655. doi:10.1145/277044.277210 (<https://doi.org/10.1145%2F277044.277210>). ISBN 978-0-89791-964-7.

6. Bhadra, Jayanta; Abadir, Magdy S.; Wang, Li-C.; Ray, Sandip (March 2007). "A Survey of Hybrid Techniques for Functional Verification". *IEEE Design & Test of Computers*. **24** (2): 112–122. Bibcode:2007IDTC...24..112B (<https://ui.adsabs.harvard.edu/abs/2007IDTC...24..112B>). doi:10.1109/MDT.2007.30 (<https://doi.org/10.1109%2FMDT.2007.30>). ISSN 1558-1918 (<https://search.worldcat.org/issn/1558-1918>).

7. A., Ismail, Khaled; Ghany, Mohamed A. Abd El (January 2021). "Survey on Machine Learning Algorithms Enhancing the Functional Verification Process" (<https://doi.org/10.3390%2Felectronics10212688>). *Electronics*. **10** (21): 2688. doi:10.3390/electronics10212688 (<https://doi.org/10.3390%2Felectronics10212688>). ISSN 2079-9292 (<https://search.worldcat.org/issn/2079-9292>).

8. O, Emma; Packwood, Jack; Oistein, Michael. "Future Trends in ASIC Design Verification: The Convergence of Machine Learning and Hardware Security for AI Systems" (<https://www.researchgate.net/publication/391147791>). *researchgate.net*.

Retrieved from "https://en.wikipedia.org/w/index.php?title=Functional_verification&oldid=1374145528"

https://en.wikipedia.org/wiki/Functional_verification

2/2